EXERCICIO C

Aluno:Ruan Silva Carvalho

Exercício C1 - Taxa de Sinal

#include <stdio.h>

double calcular_taxa_sinal(double c, double N, double r) { return c * N * (1.0 / r); }

int main(void) { double c, N, r;

```
printf("Digite o fator de caso (c): ");
scanf("%lf", &c);

printf("Digite a taxa de dados N em bps (ex: 1000000 para 1 Mbps): ");
scanf("%lf", &N);

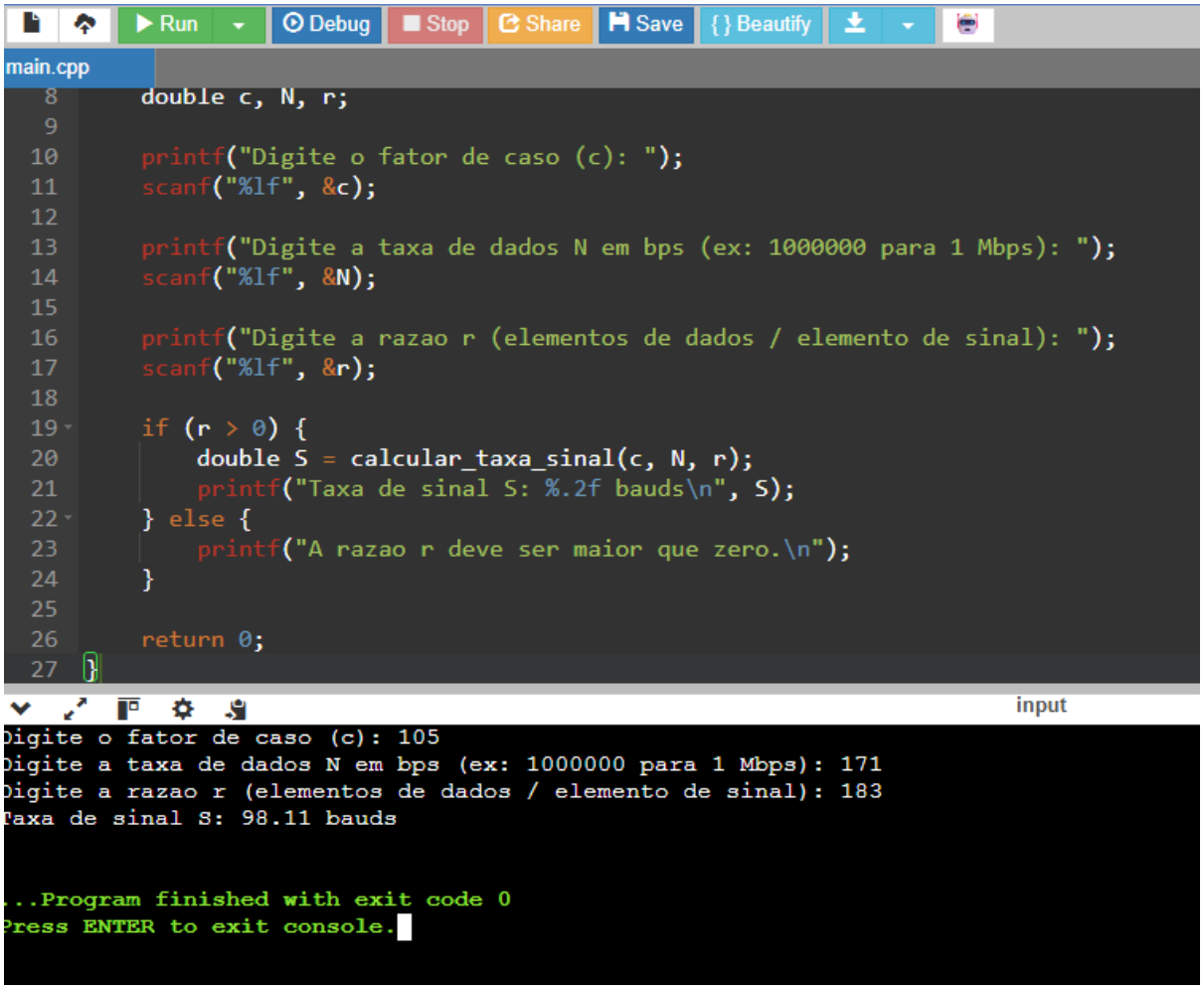
printf("Digite a razao r (elementos de dados / elemento de sinal): ");
scanf("%lf", &r);

if (r > 0) {
    double S = calcular_taxa_sinal(c, N, r);
    printf("Taxa de sinal S: %.2f bauds\n", S);
} else {
    printf("A razao r deve ser maior que zero.\n");
}

return 0;


}
```

Exercício C1: Este calcula a **taxa de sinal (S)** em bauds. Basicamente, pega no fator de caso (c), multiplica pela taxa de dados em bps (N) e divide pela razão (r) entre os elementos de dados e os elementos de sinal ($S = c \times N \times (1/r)$).



Exercício C2 - Bits Extras por Deriva de Clock

```
#include <stdio.h>


int main(void) {
   double taxa_dados, desvio_percentual;


   printf("Digite a taxa de dados em bps: ");
   scanf("%lf", &taxa_dados);


   printf("Digite o desvio percentual do clock (ex: 0.2 para 0.2%%): ");
   scanf("%lf", &desvio_percentual);

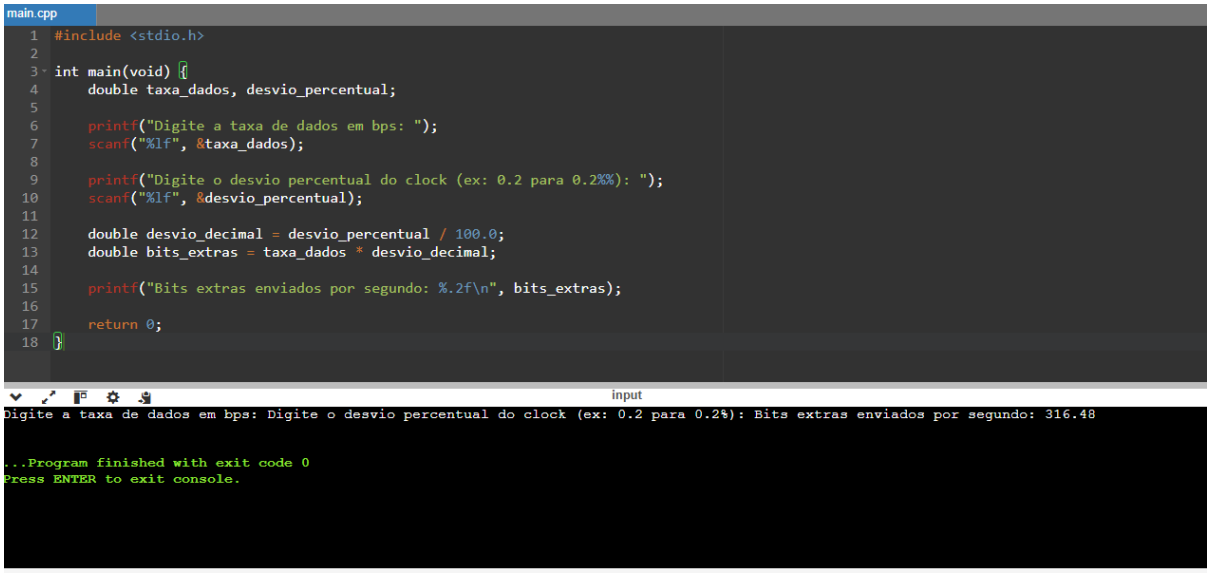

   double desvio_decimal = desvio_percentual / 100.0;
```

```
    double bits_extras = taxa_dados * desvio_decimal;


    printf("Bits extras enviados por segundo: %.2f\n", bits_extras);


    return 0;
}
```

Exercício C2: Este calcula os **bits extra devido à deriva de clock**. Se o relógio do emissor for um bocadinho mais rápido do que o do recetor (por exemplo, $0,2\%$ a mais), ele vai enviar bits a mais a cada segundo. O programa multiplica a taxa de dados por essa percentagem de desvio para nos dar exatamente quantos bits a mais aparecem.



Exercício C3 - Taxa de Amostragem de Nyquist

```
#include <stdio.h>


double nyquist(double f_min, double f_max, int passa_baixa) {
    // Tanto para passa-baixa quanto para passa-faixa no critério de Nyquist básico do livro,
    // a taxa mínima de amostragem depende do dobro da frequência máxima (2 * f_max).
    (void)f_min; // Evita aviso de variável não utilizada caso seja passa-baixa
    if (passa_baixa) {
        return 2.0 * f_max;
    } else {
        return 2.0 * f_max; // Frequência máxima dita o teto para passa-faixa também
```

```
    }
}

int main(void) {
    double f_min, f_max;
    int tipo;

    printf("Digite a frequencia minima (Hz) [0 se passa-baixa]: ");
    scanf("%lf", &f_min);

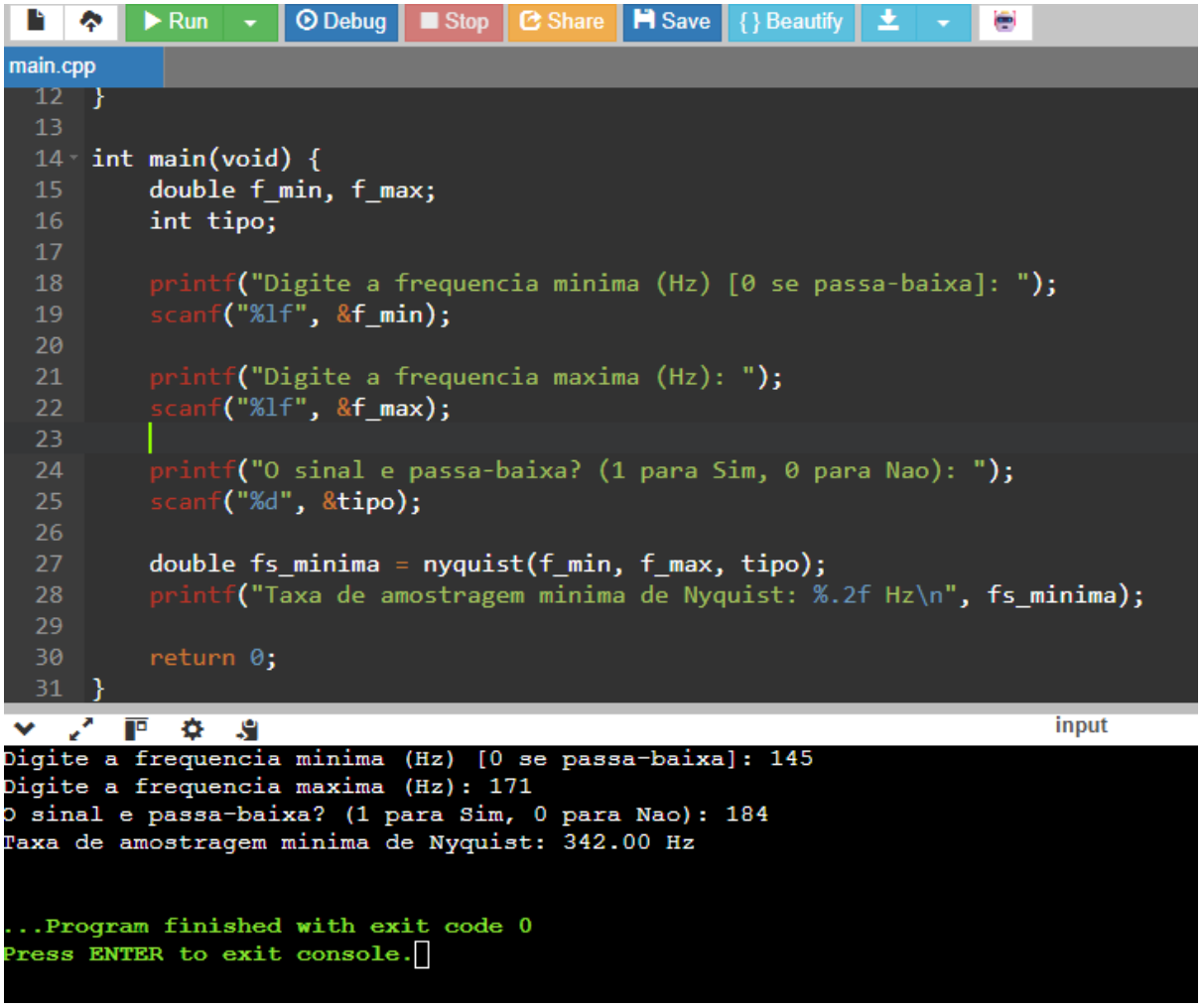
    printf("Digite a frequencia maxima (Hz): ");
    scanf("%lf", &f_max);

    printf("O sinal e passa-baixa? (1 para Sim, 0 para Nao): ");
    scanf("%d", &tipo);

    double fs_minima = nyquist(f_min, f_max, tipo);
    printf("Taxa de amostragem minima de Nyquist: %.2f Hz\n", fs_minima);

    return 0;
}
```

Exercício C3: Este aplica o **Teorema de Nyquist**. Lê a frequência máxima de um sinal e calcula a taxa de amostragem mínima necessária para o reconstruir sem perdas, que é simplesmente o dobro dessa frequência máxima ($2 \times f_{\text{max}}$)



Exercício C4 - Bit Rate e SNRdB de um Sinal PCM

#include <stdio.h> #include <math.h>

int main(void) { double largura_banda; int L;

```
printf("Digite a largura de banda do sinal analogico (Hz): ");
scanf("%lf", &largura_banda);

printf("Digite o numero de niveis de quantizacao L: ");
scanf("%d", &L);

if (L > 0 && largura_banda > 0) {
    // nb = log2(L) = log(L) / log(2)
    double nb = log((double)L) / log(2.0);

    // Pelo teorema de Nyquist, a taxa de amostragem fs = 2 * B
    // Bit rate N = nb * fs = nb * 2 * B
    double bit_rate = nb * 2.0 * largura_banda;
```

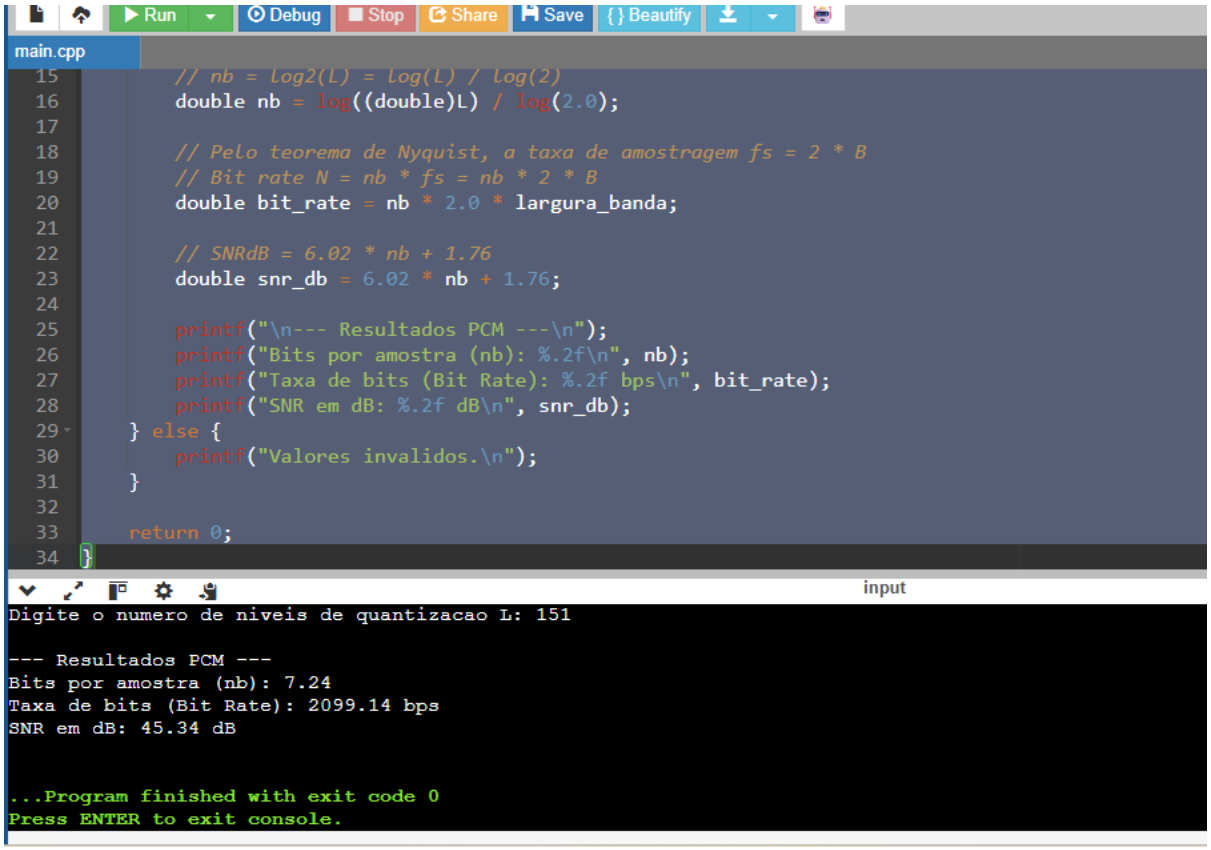
```
    // SNRdB = 6.02 * nb + 1.76
    double snr_db = 6.02 * nb + 1.76;

    printf("\n--- Resultados PCM ---\n");
    printf("Bits por amostra (nb): %.2f\n", nb);
    printf("Taxa de bits (Bit Rate): %.2f bps\n", bit_rate);
    printf("SNR em dB: %.2f dB\n", snr_db);
} else {
    printf("Valores invalidos.\n");
}

return 0;


}
```

Exercício C4: Este foca-se no **PCM (*Pulse Code Modulation*)**. Lê a largura de banda analógica e o número de níveis de quantização (L), descobre quantos bits leva cada amostra ($n_b = \log_2(L)$), calcula a taxa de bits e ainda determina o SNR_{dB} através da fórmula $6{,}02 \times n_b + 1{,}76$.



Exercício C5 - Overhead Síncrono vs. Assíncrono

#include <stdio.h>

```
int main(void) {
   long long num_caracteres;
   int bits_por_caractere;

   printf("Digite o numero de caracteres: ");
   scanf("%lld", &num_caracteres);

   printf("Digite o numero de bits por caractere (ex: 8): ");
   scanf("%d", &bits_por_caractere);

   // Modo Síncrono: apenas os dados puros (sem bits extras por caractere)
   long long bits_sincrono = num_caracteres * bits_por_caractere;
   double redundancia_sincrono = 0.0; // 0% de overhead na transmissão básica de dados puros

   // Modo Assíncrono: adiciona 1 bit de start e 1 bit de stop por caractere (total = 2 bits extras)
   int bits_extras_assincrono = 2;
   long long total_bits_assincrono = num_caracteres * (bits_por_caractere + bits_extras_assincrono);

   // Percentual de redundância assíncrona = (bits extras / total) * 100
   long long total_bits_extras = num_caracteres * bits_extras_assincrono;
   double redundancia_assincrono = ((double)total_bits_extras / total_bits_assincrono) * 100.0;

   printf("\n--- Comparativo de Transmissao ---\n");
   printf("Modo Sincrono -> Total de bits: %lld | Redundancia: %.2f%%\n", bits_sincrono, redundancia_sincrono);
   printf("Modo Assincrono -> Total de bits: %lld | Redundancia: %.2f%%\n", total_bits_assincrono, redundancia_assincrono);
```

```
    return 0;
}
```

Exercício C5: Este compara o **overhead entre a transmissão síncrona e a assíncrona**. No modo síncrono enviamos os dados de forma contínua, enquanto no assíncrono adicionamos um bit de *start* e um bit de *stop* por cada caractere. O programa calcula o total de bits em ambos os cenários e diz-nos qual é a percentagem de redundância do método assíncrono

